arXiv:1805.08657v1 [cs.LG] 22 May 2018

Robust Conditional Generative Adversarial Networks

Grigorios G. Chrysos[1], **Jean Kossaifi**[1], **Stefanos Zafeiriou**[1]
[1] Department of Computing, Imperial College London, UK
{g.chrysos, jean.kossaifi, s.zafeiriou}@imperial.ac.uk

Abstract

Conditional generative adversarial networks (cGAN) have led to large improvements in the task of conditional image generation, which lies at the heart of computer vision. The major focus so far has been on performance improvement, while there has been little effort in making cGAN more robust to noise or leveraging structure in the output space of the model. The end-to-end regression (of the generator) might lead to arbitrarily large errors in the output, which is unsuitable for the application of such networks to real-world systems. In this work, we introduce a novel conditional GAN, called *RoCGAN*, which adds implicit constraints to address the issue. Our proposed model augments the generator with an unsupervised pathway, which encourages the outputs of the generator to span the target manifold even in the presence of large amounts of noise. We prove that RoCGAN shares similar theoretical properties as GAN and experimentally verify that the proposed model outperforms existing state-of-the-art cGAN architectures by a large margin in a variety of domains including images from natural scenes and faces.

1 Introduction

Image-to-image translation and more generally conditional image generation lie at the heart of computer vision. Conditional Generative Adversarial Networks (cGAN) [1] have become a dominant approach in the field, e.g. in dense[1] regression [2, 3, 4, 5, 6, 7, 8]. They accept a source signal as input, e.g. prior information in the form of an image or text, and map it to the target signal (image). Ideally, the output should be a sample from the target manifold, however the mapping of cGAN does not constrain the output and thus it can be arbitrarily off the target manifold [9]. This is a critical problem both for academic and commercial applications. If we aim to utilize cGAN or similar methods as a production technology, they need to be reliable and have performance guarantees under large amount of noise.

Both supervised regression and classification suffer from sensitivity to noise and lack of output constraints. One notable line of research consists in complementing supervision with unsupervised learning modules. The unsupervised module forms a new pathway that can be fed either the same, or different data samples. The unsupervised pathway enables the network to explore the structure that is not present in the labelled training set, while implicitly constraining the output. The addition of the unsupervised modules is only required during the training stage and results in no additional computational cost during inference. In [10, 11] the original bottom-up network is modified to include top-down modules during training. However, in dense regression both bottom-up and top-down modules exist by default, and such methods are thus not trivial to extend to the dense regression tasks.

Motivated by the combination of supervised and unsupervised pathways, we propose a novel conditional GAN which includes implicit constraints in the latent subspaces. We coin this new model '*Robust Conditional GAN*' (*RoCGAN*). In the original cGAN the generator accepts a source signal and

[1]The output includes at least as many dimensions as the input, e.g. super-resolution, or text-to-image translation.

Preprint. Work in progress.

maps it to the target domain. In our work, we (implicitly) constrain the decoder to generate samples that span only the target manifold. To that end, we replace the original generator, i.e. encoder-decoder, with a two pathway module. The first pathway, similarly to the cGAN generator, performs regression while the second is an autoencoder in the target domain (unsupervised pathway). The two pathways share a similar network structure, i.e. each one includes an encoder-decoder network. The weights of the two decoders are shared which encourages the latent representations of the two pathways to be semantically similar. Intuitively, this can be thought of as constraining the output of our dense regression to span the target subspace. The unsupervised pathway enables the utilization of all the samples in the target domain even in the absence of a corresponding input sample. During inference, the unsupervised pathway is no longer required, therefore the testing complexity remains the same as in cGAN. We prove that RoCGAN share similar theoretical properties with the original GAN, i.e. convergence and optimal discriminator. An experiment with synthetic data is designed to visualize the aforementioned subspaces and assess our intuition. Moreover, thorough experimentation with both images from natural scenes and human faces is conducted in different tasks to evaluate the model. We compare RoCGAN with both the state-of-the-art cGAN and the recent method of [12]. The experimental results demonstrate that RoCGAN outperform the baseline by a large margin in all cases.

Notation: Given a set of N samples, $\boldsymbol{s}^{(n)}$ denotes the nth conditional label, e.g. a prior image; $\boldsymbol{y}^{(n)}$ denotes the respective target image. Unless explicitly mentioned otherwise $||\cdot||$ will declare an ℓ_1 norm.

2 Related work

Conditional image generation is a popular task in computer vision, dominated by approaches similar to cGAN. Apart from cGAN, the method in [2], widely known as 'pix2pix', is the main alternative. Pix2pix includes three modifications over the baseline cGAN: i) lateral skip connections between the encoder and the decoder network are added in the generator, ii) the discriminator accepts pairs of source/gt and source/model output images, iii) additional content loss terms are added. The authors demonstrate how those performance related modifications can lead to an improved visual outcome. Despite the improved performance, the problem with the additional guarantees remains the same. That is we do not have any direct supervision in the process, since both the latent subspace and the projection are learned; the only supervision is provided by the ground-truth (gt) signal in the generator's output.

Adding regularization terms in the loss function can impose stronger supervision, thus restricting the output. The most frequent regularization term is feature matching, e.g. perceptual loss [4, 13], or embeddings for faces [14]. Feature matching encourages the projections of the generated and the ground-truth signals to match in a low-dimensional space. However, the pre-defined feature space is restrictive. The method introduced in [15] performs the feature matching in the discriminator; the motivation lies in matching the low-dimensional distributions created by the discriminator layers. Matching the discriminator's features has demonstrated empirical success. However this does not affect the generator and its latent subspaces directly.

A new line of research that correlates with our goals is that of adversarial attacks [16]. It is observed that perturbing input samples with a small amount of noise, often imperceptible to the human eye, can lead to severe classification errors. There are several techniques to 'defend' against such attacks. A recent such example is the Fortified networks of [17] which uses Denoising Autoencoders [18] to ensure that the input samples do not fall off the target manifold. Even though RoCGAN share similarities with those methods, the scope is different since a) the output of our method is high-dimensional[2] and b) adversarial examples are not extended to dense regression.

Except for the study of adversarial attacks, combining supervised and unsupervised learning has been used for enhancing the classification performance. In the Ladder network [10] the authors modify a typical bottom-up network for classification by adding a decoder and lateral connections between the encoder and the decoder. During training they utilize the augmented network as two pathways: i) the labelled input samples are fed to the initial bottom-up module, ii) the input samples are corrupted with noise and fed to the encoder-decoder with the lateral connections. The latter pathway is an

[2] In the classification tasks studied, e.g. the popular Imagenet [19], there are up to a thousand classes, while our output includes tens or hundreds of thousands of dimensions.

autoencoder; the idea is that it can strengthen the resilience of the network to samples outside the input manifold, while it improves the classification performance.

Our core goal consists in constraining the model's output. Aside from deep learning approaches, such constraints in manifolds were typically tackled with component analysis. Canonical correlation analysis [20] has been extensively used for finding common subspaces that maximally correlate the data [21]. The recent work of [22] combines the expressiveness of neural networks with the theoretical guarantees of classic component analysis.

3 Method

In this section, we elucidate our proposed RoCGAN. To make the paper self-contained we first review the original conditional GAN model (sec. 3.1), before introducing RoCGAN (sec. 3.2). Sequentially, we pose the modifications required in case of shortcut connections from the encoder to the decoder (sec. 3.3), we study the theoretical properties of our method (sec. 3.4). Sec. 3.5 we access the intuition behind our model with synthetic data. The core idea in RoCGAN is to constrain the model's output and perform more robust image generation, which we achieve by replacing the single pathway in the generator with two pathways.

3.1 Conditional GAN

GAN consist of a generator and a discriminator module commonly optimized with alternating gradient descent methods. The generator tries to model the target distribution p_d and sample from it; the discriminator D tries to distinguish between the samples generated from the model and the target (ground-truth) distributions. Conditional GAN (cGAN) [1] extend the formulation by providing the generator with additional labels. In cGAN the generator G typically takes the form of an encoder-decoder network, where the encoder projects the label into a low-dimensional latent subspace and the decoder performs the opposite mapping, i.e. from low-dimensional to high-dimensional subspace. If we denote $p_{\boldsymbol{z}}$ the distribution of the noise, $\boldsymbol{s}$ the conditioning label and $\boldsymbol{y}$ a sample from the target data, the adversarial loss is expressed as:

$$\begin{aligned}\mathcal{L}_{adv}(\boldsymbol{G},\boldsymbol{D}) = \mathbb{E}_{\boldsymbol{s},\boldsymbol{y}\sim p_d(\boldsymbol{s},\boldsymbol{y})}[\log \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s})] + \\ \mathbb{E}_{\boldsymbol{s}\sim p_d(\boldsymbol{s}),\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log(1-\boldsymbol{D}(\boldsymbol{G}(\boldsymbol{s},\boldsymbol{z})|\boldsymbol{s}))]\end{aligned} \tag{1}$$

by solving the following min-max problem:

$$\begin{aligned}\min_{\boldsymbol{w}_G}\max_{\boldsymbol{w}_D}\mathcal{L}_{adv}(\boldsymbol{G},\boldsymbol{D}) = \min_{\boldsymbol{w}_G}\max_{\boldsymbol{w}_D}\mathbb{E}_{\boldsymbol{s},\boldsymbol{y}\sim p_d(\boldsymbol{s},\boldsymbol{y})}[\log \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s},\boldsymbol{w}_D)] + \\ \mathbb{E}_{\boldsymbol{s}\sim p_d(\boldsymbol{s}),\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log(1-\boldsymbol{D}(\boldsymbol{G}(\boldsymbol{s},\boldsymbol{z}|\boldsymbol{w}_G)|\boldsymbol{s},\boldsymbol{w}_D))]\end{aligned}$$

where $\boldsymbol{w}_G, \boldsymbol{w}_D$ denote the generator's and the discriminator's parameters respectively. To simplify the notation, we drop the dependencies on the parameters and the noise $\boldsymbol{z}$ in the rest of the paper.

The works of [15, 2] demonstrate that additional loss terms improve both the convergence and the final outcome. In our experimentation we verified the improvement, hence we consider those as part of the baseline. Apart from the content loss, the feature matching loss [15] is included:

$$\mathcal{L}_f = \sum_{n=1}^{N} ||\pi(\boldsymbol{G}(\boldsymbol{s}^{(n)})) - \pi(\boldsymbol{y}^{(n)})|| \tag{2}$$

where $\pi()$ extracts the features from the penultimate layer of the discriminator.

The final loss function for the cGAN is the following:

$$\mathcal{L} = \mathcal{L}_{adv} + \lambda_c \cdot \underbrace{\sum_{n=1}^{N} ||\boldsymbol{G}(\boldsymbol{s}^{(n)}) - \boldsymbol{y}^{(n)}||}_{content-loss} + \lambda_\pi \cdot \mathcal{L}_f \tag{3}$$

where λ_c, λ_π are hyper-parameters to balance the loss terms.

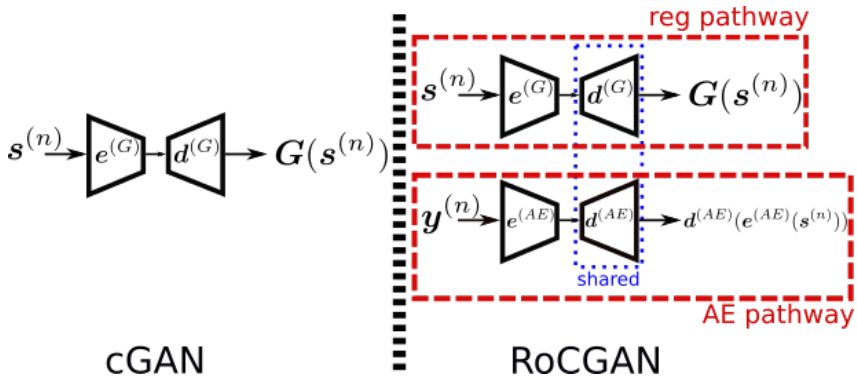


Figure 1: Schematic of the generator of cGAN versus our proposed RoCGAN. The single pathway of the original model is replaced with two pathways.

3.2 RoCGAN

Just like cGAN, RoCGAN consist of a generator and a discriminator. The generator of RoCGAN includes two pathways instead of the single pathway of the original cGAN. The first pathway, alleged *reg pathway* henceforth, performs a similar regression as its counterpart in cGAN; it accepts a sample from the source domain and maps it to the target domain. We introduce an additional unsupervised pathway, named *AE pathway*. AE pathway works as an autoencoder in the target domain. Both pathways have similar architectures, that is they both consist of encoder-decoder networks. The two pathways share the weights of their decoders, which encourages the outputs of the regression to span the target manifold and not induce arbitrarily large errors. A schematic of the generator is illustrated in Fig. 1. The discriminator can remain the same as the cGAN: it accepts the reg pathway's output along with the corresponding target sample as input.

The unsupervised module (autoencoder in the target domain) contributes the following loss term:

$$\mathcal{L}_{AE} = \sum_{n=1}^{N} [f_d(\boldsymbol{y}^{(n)}, \boldsymbol{d}^{(AE)}(\boldsymbol{e}^{(AE)}(\boldsymbol{y}^{(n)})))] \tag{4}$$

where f_d denotes a function used to measure the divergence (in this work an ℓ_1 loss); $\boldsymbol{e}^{(AE)}, \boldsymbol{d}^{(AE)}$ symbolize the encoder and the decoder of the AE pathway respectively. The superscript 'AE' abbreviates modules of the AE pathway and 'G' modules of the reg pathway.

By sharing the weights of the decoders, we encourage the latent representations to be semantically similar, which we enforce further by adding an additional loss term in the latent representations. This term minimizes the distance between the encoders' outputs, i.e. it encourages them to not only be semantically similar but also spatially close (in the subspace spanned by the generator). The latent loss term is:

$$\mathcal{L}_{lat} = \sum_{n=1}^{N} ||\boldsymbol{e}^{(G)}(\boldsymbol{s}^{(n)}) - \boldsymbol{e}^{(AE)}(\boldsymbol{y}^{(n)})|| \tag{5}$$

The final loss function of RoCGAN combines the loss terms of the original cGAN (eq. 3) with the additional two terms for the AE pathway:

$$\mathcal{L}_{RoCGAN} = \mathcal{L}_{adv} + \lambda_c \cdot \underbrace{\sum_{n=1}^{N} ||\boldsymbol{G}(\boldsymbol{s}^{(n)}) - \boldsymbol{y}^{(n)}||}_{content-loss} + \lambda_\pi \cdot \mathcal{L}_f + \lambda_{ae} \cdot \mathcal{L}_{AE} + \lambda_l \cdot \mathcal{L}_{lat} \tag{6}$$

with λ_{ae}, λ_l the regularization hyper-parameters, optimized on the validation set.

3.3 RoCGAN with skip connections

The RoCGAN of sec. 3.2 describes a family of networks and not a pre-defined set of layers. A special case of RoCGAN emerges when skip connections are included in the generator. In the next few paragraphs, we study the modification required, i.e. an additional loss term.

Skip connections are frequently used as they enable deeper layers to capture more abstract representations without the need of memorizing all the information. Nevertheless, the effects of the skip connections in the representation space have not been thoroughly studied. The lower-level representations are propagated directly to the decoder through the shortcut, which makes it harder to train the longer path [10], i.e. the network excluding the skip connections.

This challenge can be implicitly tackled by maximizing the variance captured by the longer path representations. To that end, we add a loss term that penalizes the correlations in the representations (of a layer) and thus implicitly encourage the representations to capture diverse and useful information. We implement the decov loss introduced in [23]:

$$\mathcal{L}_{decov} = \frac{1}{2}(||\boldsymbol{C}||_F^2 - ||diag(\boldsymbol{C})||_2^2) \tag{7}$$

where $diag()$ computes the diagonal elements of a matrix and $\boldsymbol{C}$ is the covariance matrix of the layer's representations. The loss is minimized when the covariance matrix is diagonal, i.e. it imposes a cost to minimize the covariance of hidden units without restricting the diagonal elements that include the variance of the hidden representations. The decov loss is applied only in the output of the encoder, as this is sufficient to minimize the correlations in the longer path.

A similar loss is explored in [24], where the decorrelation loss is applied in every layer. Their loss term has stronger constraints: i) it favors an identity covariance matrix but also ii) penalizes the smaller eigenvalues of the covariance more. The assumptions, e.g. that the average activation is zero, along with the overhead introduced by computing the eigenvalues of the covariance consist the loss of [24] prohibitive for our case.

3.4 Theoretical analysis

In the next few paragraphs, we perform a theoretical analysis and prove that RoCGAN share the properties of the original GAN [25]. We derive the optimal discriminator and then compute the optimal value of $\mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D})$.

Proposition 1. *If we fix the generator $\boldsymbol{G}$ (reg pathway), the optimal discriminator is:*

$$\boldsymbol{D}^* = \frac{p_d(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})} \tag{8}$$

where p_g is the model (generator) distribution.

Proof. The proof is a trivial extension of the proof in [25]; the details are deferred to the appendix. □

Proposition 2. *Given the optimal discriminator $\boldsymbol{D}^*$ the global minimum of $\mathcal{L}_{adv}$ is reached if and only if $p_g = p_d$, i.e. when the model (generator) distribution matches the data distribution.*

Proof. The full proof is in the appendix. □

3.5 Experiment on synthetic data

We design an experiment on synthetic data to explore the differences between the baseline generator and our modified generator. Specifically, we design a network where each encoder/decoder consists of a single fully connected layer followed by a RELU. We optimize the generators only, to avoid adding extra learned parameters.

The inputs/outputs of this network are coordinates in a low-dimensional space. The input vector is $[x + 1, x^2, e^{-x^2}]$ with $x \in [-1, 1]$ uniformly sampled; the output vector is

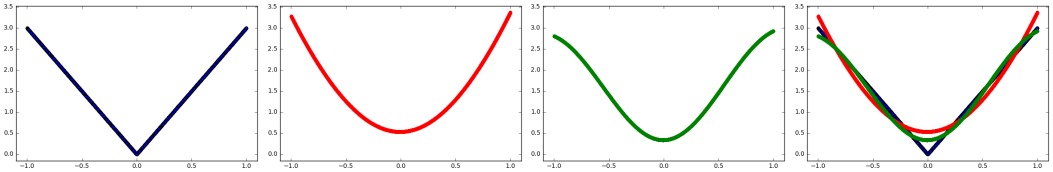


Figure 2: Qualitative results in the synthetic experiment of sec. 3.5. From left to right: The target (ground-truth) curve, the output of the single pathway network (baseline), the two pathway network and all three overlaid. We notice that even though the two networks have the same parameters during inference, our proposed network is able to localize the points more precisely.

$[3|x|, e^x, \frac{1}{\pi \cdot sigma^2} \cdot (1-k)e^{-k} + 1]$ with $k = \frac{x^2}{2 \cdot sigma^2}$. The last output term is the Ricker wavelet ($sigma = 0.25$). The network accepts the three element vector, projects it to a two dimensional space and the decoder maps it to the target space which is also three dimensional.

We train the baseline and the autoencoder modules independently and use their pre-trained weights to initialize the two pathway network. The loss function of the two pathway network consists of the $\mathcal{L}_{lat}$ (eq. 5) and ℓ_2 content losses in the two pathways. The networks are trained either till convergence or till $100,000$ iterations (batch size 128) are completed.

During testing, $6,400$ new points are sampled and the resulting figure for the first element of the output vector ($3|x|$) is visualized in Fig. 2. The ℓ_1 errors for the two cases are: $81,915$ for the baseline and $73,887$ for the two pathway generator. We notice that the two pathway generator approximates the target manifold better with the same number of parameters during inference time.

4 Experiments

We experiment with two categories of images with significant applications: images from i) natural scenes and ii) faces. In the natural scenes case, we constrain the number of training images to few thousand since frequently that is the scale of the labelled examples available. Additional training details and experimental results are deferred to the appendix.

We use two baseline models each of which is then modified to our proposed method[3]. Henceforth a 'layer' refers to a block of three units: a convolutional unit with a 4×4 kernel size, followed by Leaky RELU and batch normalization [26]. The first baseline consists of four layers in the encoder followed by four layers in the decoder. We dump this model as '4layer'. To further assess our proposed method in larger networks, we implement another baseline with five layers in each of the encoder-decoder, which we name '5layer'. Both the '4layer' and the '5layer' are used as baselines and enable us to test the proposed method in case of both limited capacity and larger representational power.

The networks for our method are obtained by augmenting the baseline models as follows: first we duplicate the encoder/decoder and share the decoder's weights in the two pathways. In addition, we modify the loss function by adding the latent loss and the reconstruction loss for the AE pathway. More elaborate extensions of the baseline, such as different number of encoder layers or latent loss in part of the representation [27], are left for future work.

Two inverse tasks, i.e. denoising and sparse inpainting, are selected for our quantitative evaluation. During training, the images are corrupted, for the two tasks, in the following way: for denoising 25% of the pixels in each channel are uniformly dropped; for sparse inpainting 50% of the pixels are converted to black. During testing, we evaluate the methods in two settings: i) similar corruption as they were trained, ii) more intense corruption, i.e. we drop 35% of the pixels in the denoising case and 75% of the pixels in the sparse inpainting case. For the evaluation we employ two error metrics, i.e. i) classic ℓ_1 loss in the pixel space, ii) an image quality loss (SSIM) [28]. We train and test our method against the i) baseline cGAN, ii) the recent OneNet [12]. OneNet is the state-of-the-art method in inverse tasks and uses an ADMM learned prior, i.e. it projects the corrupted prior images into the subspace of natural images to guide the ADMM solver.

[3] Additional information about the network can be found in the appendix material.

4.1 Natural scenes

We train the '4layer' baseline/RoCGAN with images from natural scenes, both indoors and outdoors. The $4,900$ samples of the VOC 2007 Challenge [29] form the training set, while the $10,000$ samples of tiny ImageNet [19] consist the testing set.

The quantitative evaluation with SSIM is presented in Tab. 1. In both inverse tasks RoCGAN improve the baseline cGAN results by a margin of 0.05 ($10-13\%$ relative improvement). When we apply additional corruption in the testing images, RoCGAN are more robust with a considerable improvement over the baseline. This can be attributed to the implicit constraints of the AE pathway, i.e. the decoder is more resilient to approximating the target manifold samples.

Obj. / Task / *Method*	Faces				Natural Scenes			
	Denoising		Sparse Inpaint.		Denoising		Sparse Inpaint.	
	25%	35%	50%	75%	25%	35%	50%	75%
[12]	0.758	0.748	0.701	0.682	0.591	0.574	0.585	0.535
Baseline-4layer	0.803	0.765	0.801	0.701	0.628	0.599	0.639	0.542
Ours-4layer	0.832	0.815	0.804	0.708	0.668	0.654	0.648	0.548

Table 1: Quantitative results in the '4layer' network in both faces and natural scenes cases. For both 'objects' we compute the SSIM, while the respective results with ℓ_1 are in the appendix. In both denoising and sparse inpainting, the leftmost evaluation is the one with corruptions similar to the training, while the one on the right consists of samples with additional corruptions, e.g. in denoising 35% of the pixels are dropped.

Task / *Method*	Denoising		Sparse Inpaint.	
	25%	35%	50%	75%
Baseline-5layer	0.851	0.826	0.819	0.707
Ours-5layer	0.890	0.884	0.873	0.818
Baseline-4layer-50k	0.788	0.747	0.798	0.617
Ours-4layer-50k	0.829	0.813	0.813	0.681
Baseline-4layer-skip	0.885	0.863	0.855	0.726
Ours-4layer-skip	0.892	0.878	0.859	0.744

Table 2: Additional quantitative results for the following protocols: i) '5layer' network, ii) 50 thousand training images, iii) skip connections. The protocols are described in sec. 4.2.

4.2 Faces

In this experiment we utilize the MS-Celeb [30] as the training set, and the whole Celeb-A [31] as the testing set ($202,500$ samples). Both the training and the testset contain hundreds of thousands of images, which we utilize for different training protocols.

In the first protocol, we use three million images from MS-Celeb as the training set and train both the '4layer' and '5layer' networks. The results of the quantitative evaluation exist in the tables 1 and 2 respectively. Note that in '5layer' networks the results are improved due to the larger capacity of the network. In both the '4layer' and '5layer' cases RoCGAN outperform both the baseline cGAN and the OneNet by a significant margin. The difference becomes even more pronounced in the cases of extreme corruptions where up to $\mathbf{15}\%$ relative performance improvement is observed (sparse inpainting case).

We now assess whether RoCGAN can benefit from additional unlabelled samples in the target domain. To do so, we randomly pick $50,000$ images for training and three million (target) images without label for the unsupervised part. We then train RoCGAN by feeding the AE pathway the 3 million images while the reg pathway only sees the $50,000$ training images. The results of this experiment (Table. 2 - rows 'Baseline-4layer-50k' and 'Ours-4layer-50k'), demonstrate that RoCGAN can benefit greatly

(a) (b) (c) (d) (e) (f)

Figure 3: Qualitative results; *best viewed in color*. The first row depicts the ground-truth image, the second row the corrupted one (used as input to the methods). The third depicts the output of the baseline cGAN, while the outcome of our method is illustrated in the fourth row. There are different evaluations visualized for faces: (a) Denoising, (b) denoising with additional noise at test time, (c) sparse inpainting, (d) sparse inpainting with 75% black pixels. For natural scenes the columns (e) and (f) denote the denoising and sparse inpainting results respectively.

from additional examples in the target domain. We hypothesize that this enables the autoencoder to learn a more accurate representation, which is reflected to the final RoCGAN training stage.

We also consider the case of skip connections from the encoder to the decoder. We modify the baseline '4layer' to include a skip connection from the output of the third encoder layer to the input of the second decoder layer[3]. The related results in Tab. 2 are marked as 'Baseline-4layer-skip' and 'Ours-4layer-skip' and verify our previous deductions, i.e. that RoCGAN outperform the baseline cGAN.

5 Conclusion

We introduced the Robust Conditional GAN (RoCGAN) method, a new conditional GAN capable of leveraging unsupervised data to learn better latent representations, even in the face of large amount of noise. RoCGAN's generator is composed of two pathways. The first pathway (*reg pathway*), performs the regression from the source to the target domain. The new, added pathway (*AE pathway*) is an autoencoder in the target domain. By adding weight sharing between the two decoders, we implicitly constrain the reg pathway to output images that span the target manifold. We provide a linear analogy (in the appendix) and demonstrate how RoCGAN can create more robust results, while we prove that it shares similar convergence properties with GAN [25]. We experimentally demonstrate on both synthetic data, and with images from natural scenes and faces that RoCGAN outperform existing, state-of-the-art conditional GAN architectures.

6 Acknowledgements

We would like to thank Markos Georgopoulos for our fruitful conversations during the preparation of this work.

References

[1] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.

[2] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[3] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2536–2544, 2016.

[4] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[5] Konstantinos Bousmalis, George Trigeorgis, Nathan Silberman, Dilip Krishnan, and Dumitru Erhan. Domain separation networks. In *Advances in neural information processing systems (NIPS)*, pages 343–351, 2016.

[6] Ming-Yu Liu, Thomas Breuel, and Jan Kautz. Unsupervised image-to-image translation networks. In *Advances in neural information processing systems (NIPS)*, pages 700–708, 2017.

[7] Jiahui Yu, Zhe Lin, Jimei Yang, Xiaohui Shen, Xin Lu, and Thomas S Huang. Generative image inpainting with contextual attention. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[8] Orest Kupyn, Volodymyr Budzan, Mykola Mykhailych, Dmytro Mishkin, and Jiri Matas. Deblurgan: Blind motion deblurring using conditional adversarial networks. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[9] Rene Vidal, Joan Bruna, Raja Giryes, and Stefano Soatto. Mathematics of deep learning. *arXiv preprint arXiv:1712.04741*, 2017.

[10] Antti Rasmus, Mathias Berglund, Mikko Honkala, Harri Valpola, and Tapani Raiko. Semi-supervised learning with ladder networks. In *Advances in neural information processing systems (NIPS)*, pages 3546–3554, 2015.

[11] Yuting Zhang, Kibok Lee, and Honglak Lee. Augmenting supervised neural networks with unsupervised objectives for large-scale image classification. In *International Conference on Machine Learning (ICML)*, pages 612–621, 2016.

[12] JH Rick Chang, Chun-Liang Li, Barnabas Poczos, BVK Vijaya Kumar, and Aswin C Sankaranarayanan. One network to solve them all–solving linear inverse problems using deep projection models. In *IEEE Proceedings of International Conference on Computer Vision (ICCV)*, pages 5888–5897, 2017.

[13] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *Proceedings of European Conference on Computer Vision (ECCV)*, pages 694–711, 2016.

[14] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–823, 2015.

[15] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *Advances in neural information processing systems (NIPS)*, pages 2234–2242, 2016.

[16] Xiaoyong Yuan, Pan He, Qile Zhu, Rajendra Rana Bhat, and Xiaolin Li. Adversarial examples: Attacks and defenses for deep learning. *arXiv preprint arXiv:1712.07107*, 2017.

[17] Alex Lamb, Jonathan Binas, Anirudh Goyal, Dmitriy Serdyuk, Sandeep Subramanian, Ioannis Mitliagkas, and Yoshua Bengio. Fortified networks: Improving the robustness of deep networks by modeling the manifold of hidden representations. *arXiv preprint arXiv:1804.02485*, 2018.

[18] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *International Conference on Machine Learning (ICML)*, pages 1096–1103, 2008.

[19] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *IEEE Proceedings of International Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[20] Harold Hotelling. Relations between two sets of variates. *Biometrika*, 28(3/4):321–377, 1936.

[21] Yannis Panagakis, Mihalis A Nicolaou, Stefanos Zafeiriou, and Maja Pantic. Robust correlated and individual component analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence (T-PAMI)*, 38(8):1665–1678, 2016.

[22] Calvin Murdock, Ming-Fang Chang, and Simon Lucey. Deep component analysis via alternating direction neural networks. *arXiv preprint arXiv:1803.06407*, 2018.

[23] Michael Cogswell, Faruk Ahmed, Ross Girshick, Larry Zitnick, and Dhruv Batra. Reducing overfitting in deep networks by decorrelating representations. In *International Conference on Learning Representations (ICLR)*, 2016.

[24] Harri Valpola. From neural pca to deep unsupervised learning. In *Advances in Independent Component Analysis and Learning Machines*, pages 143–171. 2015.

[25] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems (NIPS)*, 2014.

[26] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.

[27] Brian Cheung, Jesse A Livezey, Arjun K Bansal, and Bruno A Olshausen. Discovering hidden factors of variation in deep networks. In *International Conference on Learning Representations Workshops*, 2015.

[28] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions in Image Processing (TIP)*, 13(4):600–612, 2004.

[29] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision (IJCV)*, 88(2):303–338, 2010.

[30] Y. Guo et al. Ms-celeb-1m: A dataset and benchmark for large-scale face recognition. In *Proceedings of European Conference on Computer Vision (ECCV)*, pages 87–102, 2016.

[31] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *IEEE Proceedings of International Conference on Computer Vision (ICCV)*, pages 3730–3738, 2015.

[32] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence (T-PAMI)*, 35(8):1798–1828, 2013.

A Introduction

This is the appendix material accompanying the paper 'RoCGAN: Robust Conditional GAN'.

In this section, we include additional insights and the the detailed theoretical analysis along with additional experimental details. The sections are organized as following:

- In sec. B we include an insight for RoCGAN through the linear equivalent.
- The proofs for the theoretical analysis are provided in sec. C.
- Further experimental details are provided in sec. D.

Additional visualizations are provided in Fig. 5, while the tables 6 and 7 contain the quantitative results with ℓ_1 loss.

B Linear generator analogy

Even though an analysis with recent deep networks is complicated, we can provide the linear equivalent of our proposed method. As aforementioned, the discriminator in RoCGAN can remain the same as in the baseline cGAN, hence we only optimize for the generators in this section. To perform the analysis on the linear equivalent, we simply drop the piecewise non-linear units in the generators.

To convert the network to its linear equivalent, we focus on the generator as the discriminator is used only for the optimization and we do not use the piecewise non-linear units.

We assume a network with two encoding and two decoding layers in this section; all layers include only linear units. The symbols $\boldsymbol{W}_l^{(G)}$ with $l \in [1, 4]$ are the lth layer's parameters. The linear autoencoder (AE) has a similar structure; $\boldsymbol{W}_l^{(AE)}$ denote the respective parameters for the AE. We denote with $\boldsymbol{X}$ the input, with $\boldsymbol{Y}$ the (ground-truth) output and $\hat{\boldsymbol{Y}}$ the AE output, $\tilde{\boldsymbol{Y}}$ the regression output:

$$\hat{\boldsymbol{Y}} = \underbrace{\boldsymbol{W}_4^{(AE)}\boldsymbol{W}_3^{(AE)}}_{\boldsymbol{U}_D^T}\underbrace{\boldsymbol{W}_2^{(AE)}\boldsymbol{W}_1^{(AE)}}_{\boldsymbol{U}_E}\boldsymbol{Y} \tag{9}$$

is the reconstruction of the autoencoder and

$$\tilde{\boldsymbol{Y}} = \underbrace{\boldsymbol{W}_4^{(G)}\boldsymbol{W}_3^{(G)}}_{\boldsymbol{U}_{D,(G)}^T}\underbrace{\boldsymbol{W}_2^{(G)}\boldsymbol{W}_1^{(G)}}_{\boldsymbol{U}_{E,(G)}}\boldsymbol{X} \tag{10}$$

is the regression of the generator. We define the auxiliary $\boldsymbol{U}_{D,(G)}^T = \boldsymbol{W}_4^{(G)}\boldsymbol{W}_3^{(G)}$, $\boldsymbol{U}_{E,(G)} = \boldsymbol{W}_2^{(G)}\boldsymbol{W}_1^{(G)}$, $\boldsymbol{U}_D^T = \boldsymbol{W}_4^{(AE)}\boldsymbol{W}_3^{(AE)}$ and $\boldsymbol{U}_E = \boldsymbol{W}_2^{(AE)}\boldsymbol{W}_1^{(AE)}$. Then Eq. 9 and 10 can be written as:

$$\begin{cases} \tilde{\boldsymbol{Y}} = \boldsymbol{U}_{D,(G)}^T\boldsymbol{U}_{E,(G)}\boldsymbol{X} \\ \hat{\boldsymbol{Y}} = \boldsymbol{U}_D^T\boldsymbol{U}_E\boldsymbol{Y} \end{cases} \tag{11}$$

The AE approximates under mild condition robustly the manifold of the data [32], i.e. the subspaces of $\boldsymbol{U}_D^T$, $\boldsymbol{U}_E$ are robust to noise. Given that $\boldsymbol{U}_{D,(G)} = \boldsymbol{U}_D$, we constrain the output of the generator to lie in the subspaces learned with the AE.

C Detailed theoretical analysis

In this section, we provide the proofs that were ommitted from the manuscript.

Proposition 3. *If we fix the generator $\boldsymbol{G}$ (reg pathway), the optimal discriminator is:*

$$\boldsymbol{D}^* = \frac{p_d(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})} \tag{12}$$

where p_g is the model (generator) distribution.

Proof. Since the generator is fixed, the goal of the discriminator is to maximize the $\mathcal{L}_{adv}$ where:

$$\mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D}) = \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} p_d(\boldsymbol{y}, \boldsymbol{s}) \log \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s}) \boldsymbol{dyds} + \int_{\boldsymbol{s}} \int_{\boldsymbol{z}} p_d(\boldsymbol{s}) p_z(\boldsymbol{z}) \log(1 - \boldsymbol{D}(\boldsymbol{G}(\boldsymbol{s}, \boldsymbol{z})|\boldsymbol{s})) \boldsymbol{dsdz} = \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} p_d(\boldsymbol{s}, \boldsymbol{y}) \log \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s}) \boldsymbol{dy} + p_g(\boldsymbol{s}, \boldsymbol{y}) \log(1 - \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s})) \boldsymbol{dyds} \tag{13}$$

To maximize the $\mathcal{L}_{adv}$, we need to optimize the integrand above. We note that with respect to $\boldsymbol{D}$ the integrand has the form $f(y) = a \cdot log(y) + b \cdot log(1 - y)$. The function f for $a, b \in (0, 1)$ as in our case, obtains a global maximum in $\frac{a}{a+b}$, so:

$$\mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D}) \leq \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} p_d(\boldsymbol{s}, \boldsymbol{y}) \log \boldsymbol{D}^*(\boldsymbol{y}|\boldsymbol{s}) \boldsymbol{dy} + p_g(\boldsymbol{s}, \boldsymbol{y}) \log(1 - \boldsymbol{D}^*(\boldsymbol{y}|\boldsymbol{s})) \boldsymbol{dyds} \tag{14}$$

with

$$\boldsymbol{D}^* = \frac{p_d(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})} \tag{15}$$

thus $\mathcal{L}_{adv}$ obtains the maximum with $\boldsymbol{D}^*$. □

Proposition 4. *Given the optimal discriminator $\boldsymbol{D}^*$ the global minimum of $\mathcal{L}_{adv}$ is reached if and only if $p_g = p_d$, i.e. when the model (generator) distribution matches the data distribution.*

Proof. From proposition 3, we have found the optimal discriminator as $\boldsymbol{D}^*$, i.e. the $\arg\max_{\boldsymbol{D}} \mathcal{L}_{adv}$. If we replace the optimal value we obtain:

$$\begin{aligned} \max_{\boldsymbol{D}} \mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D}) = \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} p_d(\boldsymbol{s}, \boldsymbol{y}) \log \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s}) \boldsymbol{dy} + p_g(\boldsymbol{s}, \boldsymbol{y}) \log(1 - \boldsymbol{D}(\boldsymbol{y}|\boldsymbol{s})) \boldsymbol{dyds} = \\ \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} p_d(\boldsymbol{s}, \boldsymbol{y}) \log(\frac{p_d(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})}) + p_g(\boldsymbol{s}, \boldsymbol{y}) \log(1 - \frac{p_d(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})}) \boldsymbol{dyds} = \\ \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} p_d(\boldsymbol{s}, \boldsymbol{y}) \log(\frac{p_d(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})}) + p_g(\boldsymbol{s}, \boldsymbol{y}) \log(\frac{p_g(\boldsymbol{s}, \boldsymbol{y})}{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})}) \boldsymbol{dyds} \end{aligned} \tag{16}$$

We add and subtract $\log(2)$ from both terms, which after few math operations provides:

$$\begin{aligned} \max_{\boldsymbol{D}} \mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D}) = -\int_{\boldsymbol{y}} \int_{\boldsymbol{s}} (p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})) \log(2) \boldsymbol{dyds} + \\ \int_{\boldsymbol{y}} \int_{\boldsymbol{s}} (p_d(\boldsymbol{s}, \boldsymbol{y}) \log \frac{\frac{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})}{2}}{p_d(\boldsymbol{s}, \boldsymbol{y})} + p_g(\boldsymbol{s}, \boldsymbol{y}) \log \frac{\frac{p_d(\boldsymbol{s}, \boldsymbol{y}) + p_g(\boldsymbol{s}, \boldsymbol{y})}{2}}{p_g(\boldsymbol{s}, \boldsymbol{y})}) \boldsymbol{dyds} = \\ -2 \cdot \log(2) + KL(p_d || \frac{p_d + p_g}{2}) + KL(p_g || \frac{p_d + p_g}{2}) \end{aligned} \tag{17}$$

where in the last row KL symbolizes the Kullback–Leibler divergence. The latter one can be rewritten more conveniently with the help of the Jensen–Shannon (JSD) divergence as

$$\max_{\boldsymbol{D}} \mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D}) = -\log(4) + 2 \cdot JSD(p_d || p_g) \tag{18}$$

Table 3: Details of the generator for the '4layer' network baseline. Our modified generator includes in the AE pathway the same parameters. The parameters mentioned below are valid also for the '4layer-50k' and the '4layer-skip' networks. 'Filter size' denotes the size of the convolutional filters; the last number denotes the number of output filters. BN stands for batch normalization. Conv denotes a convolutional layer, while F-Conv denotes a transposed convolutional layer with fractional-stride.

(a) Encoder

Layer	Filter Size	Stride	BN
Conv. 1	$4 \times 4 \times 64$	4	$\times$
Conv. 2	$4 \times 4 \times 128$	2	✓
Conv. 3	$4 \times 4 \times 256$	2	✓
Conv. 4	$4 \times 4 \times 512$	4	✓

(b) Decoder

Layer	Filter Size	Stride	BN
F-Conv. 1	$1 \times 1 \times 256$	4	✓
F-Conv. 2	$4 \times 4 \times 128$	2	✓
F-Conv. 3	$4 \times 4 \times 64$	2	✓
F-Conv. 4	$4 \times 4 \times 3$	4	$\times$

Table 4: Details of the generator for the '5layer' network baseline.

(a) Encoder

Layer	Filter Size	Stride	BN
Conv. 1	$4 \times 4 \times 32$	2	$\times$
Conv. 2	$4 \times 4 \times 64$	2	✓
Conv. 3	$4 \times 4 \times 128$	2	✓
Conv. 4	$4 \times 4 \times 256$	2	✓
Conv. 5	$4 \times 4 \times 768$	4	✓

(b) Decoder

Layer	Filter Size	Stride	BN
F-Conv. 1	$1 \times 1 \times 256$	4	✓
F-Conv. 2	$4 \times 4 \times 128$	2	✓
F-Conv. 3	$4 \times 4 \times 64$	2	✓
F-Conv. 4	$4 \times 4 \times 32$	2	✓
F-Conv. 5	$4 \times 4 \times 3$	2	$\times$

The Jensen–Shannon divergence is non-negative and obtains the zero value only if $p_d = p_g$. Equivalently, $\max_{\boldsymbol{D}} \mathcal{L}_{adv}(\boldsymbol{G}, \boldsymbol{D}) \geq -\log(4)$ and has a global minimum (under the constraint that the discriminator is optimal) when $p_d = p_g$.

□

D Experimental details

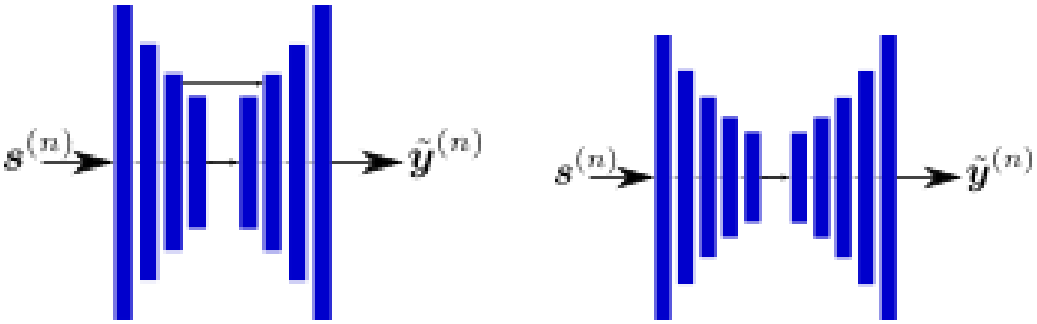
In the tables 3 and 4, the details about the layer structure for the '4layer' and '5layer' generators are provided. The discriminator retains the same structure in all the experiments in this work; the details of the discriminator structure can be found in table 5.

All the images utilized in this work are resized to $64 \times 64 \times 3$. In the case of natural scenes, instead of rescaling the images during the training stage, we crop random patches in every iteration from the image. We utilize the ADAM optimizer with a learning rate of $2 \cdot 10^{-5}$ for all our experiments. The batch size is 128 for images of faces and 64 for the natural scenes.

Table 5: Details of the discriminator. The discriminator structure remains the same throughout all the experiments in this work.

(a) Discriminator

Layer	Filter Size	Stride	BN
Conv. 1	$4 \times 4 \times 64$	2	$\times$
Conv. 2	$4 \times 4 \times 128$	2	✓
Conv. 3	$4 \times 4 \times 256$	1	✓
Conv. 4	$4 \times 4 \times 1$	1	$\times$



(a) (b)

Figure 4: The layer schematics of the generators in case of (a) the '4layer-skip' case, (b) the '5layer' case.

(a) (b) (c) (d) (e) (f) (g) (h)

Figure 5: Qualitative results; best viewed in color. The first row depicts the ground-truth image, the second row the corrupted one (input to methods), the third the output of the baseline cGAN, the fourth illustrates the outcome of our method. The four first columns are based on the protocol of '4layer' network, while the four rightmost columns on the protocol '4layer-50k'. There are different evaluations visualized for faces: (a), (e) Denoising, (b), (f) denoising with augmented noise at test time, (c), (g) sparse inpainting, (d), (h) sparse inpainting with 75% black pixels.

Obj. / Task / *Method*	Faces				Natural Scenes			
	Denoising		Sparse Inpaint.		Denoising		Sparse Inpaint.	
	25%	35%	50%	75%	25%	35%	50%	75%
[12]	701.6	801.6	838.3	876.1	1005.9	1083.4	1207.1	1401.1
Baseline-4layer	652.7	728.4	618.8	846.5	983.5	1057.7	949.4	1352.6
Ours-4layer	590.3	632.1	616.1	830.0	918.8	968.9	939.4	1351.9

Table 6: Quantitative results in the '4layer' network in both faces and natural scenes cases. In this table, the ℓ_1 loss is reported. In each task, the leftmost evaluation is the one with corruptions similar to the training, while the one on the right consists of samples with additional corruptions, e.g. in denoising 35% of the pixels are dropped.

Task / *Method*	Denoising		Sparse Inpaint.	
	25%	35%	50%	75%
Baseline-5layer	520.7	573.3	582.0	826.8
Ours-5layer	454.7	472.6	492.1	620.1
Baseline-4layer-50k	662.5	743.2	656.2	1050.8
Ours-4layer-50k	587.5	623.3	624.1	950.2
Baseline-4layer-skip	440.3	501.3	487.6	765.6
Ours-4layer-skip	403.7	499.6	481.3	747.2

Table 7: Additional quantitative results (ℓ_1 loss) for the following protocols: i) '5layer' network, ii) 50 thousand training images, iii) skip connections. The protocols are detailed in the main paper.